ASSIGNMENT

Q1. Display the employee name, department name, and salary of all employees.

```
SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

Q2. Count the number of employees in each department.

```
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Q3. Display each employee along with their manager's name.

```
SELECT e.emp_name AS employee_name, m.emp_name AS manager_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Q4. Calculate the total salary paid in each department.

```
SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

Q5. Find the average salary of employees in each department.

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
```

```
GROUP BY d.dept_name;
```

Q6. Find the department with the highest average salary.

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary DESC
LIMIT 1;
```

Q7. Display employees whose salary is greater than the overall average salary.

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q8. Display employees whose salary is greater than the average salary of their own department.

```
SELECT e.emp_name, e.salary, e.dept_id
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);
```

Q9. Find the second-highest salary among all employees.

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q10. Display the employee(s) who earn the highest salary in each department.

```
SELECT emp_name, dept_id, salary
FROM employees e
```

```
WHERE salary = (SELECT MAX(salary) FROM employees WHERE dept_id = e.dept_id);
```

Q11. Rank employees within each department based on salary in descending order.

```
SELECT emp_name, dept_id, salary,
RANK() OVER(PARTITION BY dept_id ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Q12. Display the top 3 highest-paid employees in each department.

```
SELECT *
FROM (
SELECT emp_name, dept_id, salary,
ROW_NUMBER() OVER(PARTITION BY dept_id ORDER BY salary DESC) AS rn
FROM employees
)
WHERE rn <= 3;
```

Q13. Calculate the running total of salaries based on hire date.

```
SELECT emp_name, hire_date, salary,
SUM(salary) OVER(ORDER BY hire_date) AS running_total_salary
FROM employees;
```

Q14. Display each employee's salary along with the previous employee's salary based on hire date.

```
SELECT emp_name, salary,
LAG(salary) OVER(ORDER BY hire_date) AS previous_salary
FROM employees;
```

Q15. Find customers who have never placed an order.

```
SELECT c.customer_id, c.customer_name

FROM customers c

LEFT JOIN orders o ON c.customer_id = o.customer_id

WHERE o.order_id IS NULL;
```

Q16. Display the most recent order placed by each customer.

```
SELECT *

FROM orders o

WHERE order_date = (SELECT MAX(order_date) FROM orders WHERE customer_id =
o.customer_id);
```

Q17. Calculate the total sales amount generated by each customer.

```
SELECT c.customer_name, SUM(o.amount) AS total_sales

FROM customers c

JOIN orders o ON c.customer_id = o.customer_id

GROUP BY c.customer_name;
```

Q18. Find each customer's contribution percentage to total sales.

```
SELECT c.customer_name,

SUM(o.amount) AS customer_sales,

ROUND(SUM(o.amount) * 100 / (SELECT SUM(amount) FROM orders), 2) AS
contribution_percentage

FROM customers c

JOIN orders o ON c.customer_id = o.customer_id

GROUP BY c.customer_name;
```

Q19. Identify duplicate customer names in the Customers table.

```
SELECT customer_name
```

```
FROM customers
GROUP BY customer_name
HAVING COUNT(*) > 1;
```

Q20. Categorize employees into salary bands (Low, Medium, High).

```
SELECT emp_name, salary,
CASE
WHEN salary < 80000 THEN 'Low'
WHEN salary BETWEEN 80000 AND 120000 THEN 'Medium'
ELSE 'High'
END AS salary_band
FROM employees;
```

Q21. Find employees who have the same salary as another employee.

```
SELECT e1.emp_name, e2.emp_name, e1.salary
FROM employees e1
JOIN employees e2 ON e1.salary = e2.salary AND e1.emp_id < e2.emp_id;
```

Q22. Display customers who have placed more than 3 orders.

```
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING COUNT(o.order_id) > 3;
```

Q23. Find the month with the highest total sales amount.

```
SELECT EXTRACT(MONTH FROM order_date) AS month_no,
```

```
SUM(amount) AS total_sales

FROM orders

GROUP BY EXTRACT(MONTH FROM order_date)

ORDER BY total_sales DESC

FETCH FIRST 1 ROW ONLY;
```

Q24. Calculate the number of days between consecutive orders for each customer.

```
SELECT customer_id, order_id, order_date,

order_date - LAG(order_date) OVER(PARTITION BY customer_id ORDER BY order_date) AS days_gap

FROM orders;
```

Q25. Find departments where no employee has been hired in the last 2 years.

```
SELECT d.dept_name

FROM departments d

LEFT JOIN employees e ON d.dept_id = e.dept_id

GROUP BY d.dept_id, d.dept_name

HAVING MAX(e.hire_date) < CURRENT_DATE - INTERVAL '2 years';
```